

But a certain conceptual beauty lies behind recursive algorithms that can aid your understanding of programming even if you don't often apply them. Recursion has a visual beauty as well. The technique is behind the amazing mathematical art of *fractals*, the self-similar shapes shown in Figure 1.

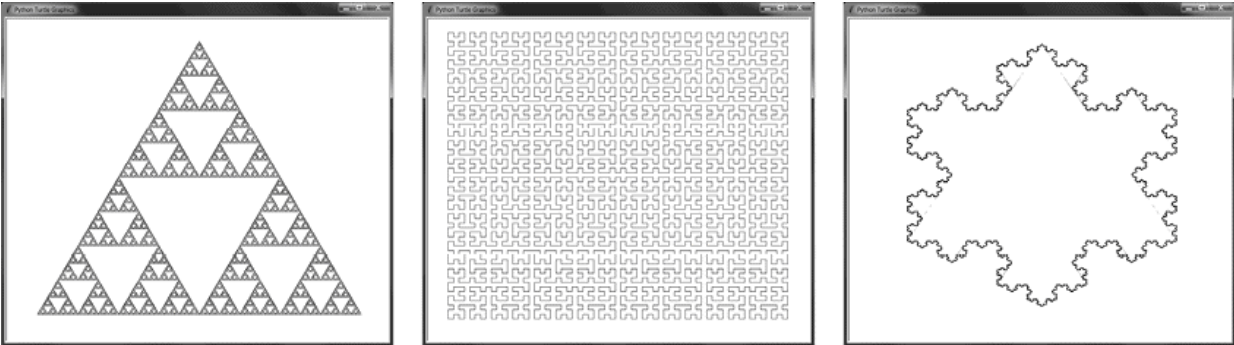


Figure 1: These examples of fractals include a Sierpiński triangle (left), a Hilbert curve (center), and a Koch snowflake (right).

However, this book is not entirely in praise of recursion. I include some sharp criticisms of this technique. Recursion is overused in cases where a simpler solution exists. Recursive algorithms can be hard to understand, have worse performance, and are susceptible to crash-causing stack overflow errors. And a certain kind of programmer may use recursion not because it's the right technique for a given problem, but simply because they feel smarter when they write code that other programmers struggle to understand. Computer scientist Dr. John Wilander once said, "When you finish a PhD in computer science, they take you to a special room and explain that you must never use recursion in real life. Its only purpose is to make programming hard for undergrads."

So, whether you want to get an edge in coding interviews, you want to create beautiful mathematical art, or you stubbornly seek to finally understand the intriguing properties of this concept, this book will be your guide down the rabbit hole that is recursion (and the rabbit holes within that rabbit hole). Recursion is one of the computer science topics that separates the professionals from the beginners. By reading this book, you'll master a great skill and learn its dark secret: recursion isn't as complicated as people think it is.